



@Academic We have a decision model in buzz (mainly Jev from Typesafe AI)

I want you to research what can be extended, is missing, or need to be added as a layer to have a firewall, governance, audit, and observability framework around it. Some references are attached.

Jev’s core innovation is **typed, calibrated decision inference that application code can consume directly**—not a governance system. That makes the gap more urgent: as its decisions become cheap enough to appear throughout a workflow, you need a vendor-neutral runtime layer that can establish whether a decision was *allowed* to become an action, on which evidence and authority, and what happened afterwards. ^{[1] [2] [3]}

The product should therefore be framed less as “Jev observability” and more as a **Decision Firewall / Decision Control Plane** that works across Jev, LLM judges, classical ML, rule engines, recommender systems, and human approvals.

What Jev provides—and does not

TypeSafe positions Jev as a System One model: a caller supplies state plus typed questions, and receives typed values, probability distributions, and—in Choice and Score—confidence. Its primitives are designed to be composed in application code: atomic judgments are combined with developer-defined logic and weights. ^[2]

That leads to a crucial boundary:

Capability	Jev / application workflow	Missing control-plane layer
Decision formation	Scores, selects, or estimates probabilities	Records the decision as a governed object
Decision logic	Developers compose atomic questions and coefficients in code	Inspects and versions the full executable decision graph
Confidence	Produces confidence/probability signals	Determines whether confidence is sufficient <i>given impact and irreversibility</i>
Type safety	Structured output conforms to schema	Validates input/evidence integrity, policy and authorization
Action	Application code branches, routes or executes	Allows, denies, delays, constrains, or escalates action
Auditability	API-level logging is possible	Produces tamper-evident, replayable, outcome-linked receipts

Capability	Jev / application workflow	Missing control-plane layer
Monitoring	Model response metrics	Measures real-world outcomes, drift, disparity, overrides and harms
Incentive integrity	Not a first-class concern	Requires disclosure, tests and monitoring of conflicts/objective distortion

“Typed” and “calibrated” reduce a category of integration error. They do **not** establish that:

- The state supplied to the model was accurate, complete, current, and authorized.
- The available options were complete and non-manipulated.
- The question encoded a legitimate objective.
- The application’s composition logic, thresholds, or downstream actions were appropriate.
- A favorable business incentive was disclosed rather than hidden inside a feature, candidate set, coefficient, or execution threshold.
- The decision was fair, legally defensible, reversible, or safe to automate.

TypeSafe explicitly recommends decomposing broad decisions into atomic questions and combining them in code with explicit coefficients. That is architecturally sensible, but it shifts a large part of the governance surface from the model into the **decision graph deployed around it**: question design, evidence mapping, weights, thresholding, orchestration, action binding, and exception handling.^[2]

The layered Decision Firewall

The recommended architecture is a mandatory mediation point between a decision producer and every consequential capability or actuator.

```

flowchart LR
    A[State, evidence, retrieval] --> B[Evidence & input integrity]
    B --> C[Decision producer: Jev / ML / LLM / rules / human]
    C --> D[Decision envelope]
    D --> E[Decision Firewall]
    E --> F{Policy, risk & authority}
    F -->|Permit| G[Capability broker / execution adapter]
    F -->|Constrain| H[Bounded action or evidence request]
    F -->|Escalate| I[Human / committee review]
    F -->|Deny| J[Safe refusal / compensating workflow]
    G --> K[Receipt, telemetry & outcome link]
    H --> K
    I --> K
    J --> K
    K --> L[Monitoring, audits & policy improvement]

```

The decisive design principle is:

The model may propose an action; only the firewall grants an executable authorization.

A logging SDK alone is useful observability, but it is not a firewall. If an agent can invoke a payment API, mutate a database, email a customer, reject an applicant, or alter access rights without going through the mediation point, the system is bypassable by design.

1. Decision contract and envelope

Define a provider-neutral, versioned **Decision Envelope**. This must be richer than choice + probability.

```
{
  "decision_id": "dec_01J...",
  "tenant_id": "acme",
  "workflow_id": "vendor-selection-v3",
  "decision_type": "vendor_recommendation",
  "risk_tier": "high",

  "objective": {
    "declared_goal": "Select supplier maximizing suitability within approved budget"
    "optimization_dimensions": [
      {"name": "technical_fit", "weight": 0.50},
      {"name": "total_cost", "weight": 0.30},
      {"name": "delivery_risk", "weight": 0.20}
    ],
    "prohibited_objectives": [
      "undisclosed_partner_preference",
      "protected-class discrimination"
    ],
    "owner": "procurement-platform"
  },

  "alternatives": [
    {"id": "vendor_a", "source": "approved_supplier_catalog"},
    {"id": "vendor_b", "source": "approved_supplier_catalog"}
  ],

  "evidence_refs": [
    {
      "evidence_id": "ev_123",
      "claim": "quoted_price",
      "source_uri": "catalog://quote/123",
      "retrieved_at": "2026-09-27T06:55:00Z",
      "integrity_hash": "sha256:...",
      "classification": "observed"
    }
  ],

  "model_assessment": {
    "provider": "typesafe",
    "model": "jev-1.13.0",
    "question_schema_version": "vendor-fit/v4",
    "state_schema_version": "vendor-state/v6",
    "selected_option": "vendor_a",
    "probabilities": {"vendor_a": 0.74, "vendor_b": 0.26},
    "confidence": 0.79,
```

```

    "model_rationale": "untrusted_claim"
  },

  "proposed_action": {
    "action_type": "create_purchase_request",
    "target": "vendor_a",
    "amount": {"currency": "USD", "value": 18500},
    "parameters_hash": "sha256:..."
  },

  "incentives": {
    "disclosed_relationships": [],
    "sponsorship_status": "unknown",
    "affiliate_commission": "none_declared"
  },

  "policy_context": {
    "policy_bundle": "procurement-policy/2026-09-15",
    "execution_authority_requested": "purchase-request-create"
  }
}

```

This separates four things that commonly get conflated:

- **Observed evidence:** source-grounded, timestamped facts.
- **Derived judgment:** model scores, classifications, probabilities, and stated rationale.
- **Normative constraints:** rules and policies defining what is permitted.
- **Execution authority:** the precise action a system is allowed to perform.

Do not treat a model's self-explanation as proof of its internal decision path. Store it as an untrusted explanation claim. The auditable basis should be the declared question, candidate set, evidence references, policy checks, workflow logic, authorization, and actual execution records.

2. Evidence integrity and provenance gateway

Before Jev sees state, validate the state. A typed input can still be factually false, stale, adversarially selected, or privacy-violating.

Required controls:

- **Evidence classification:** observed, inferred, supplied-by-user, retrieved, generated, unknown.
- **Source provenance:** source system, document/record ID, timestamp, owner, retrieval query, transformation chain, and integrity hash.
- **Freshness and expiry:** policy-enforced TTLs; e.g., do not approve a transaction using a quote older than 24 hours.
- **Completeness assertions:** required facts must be present, or missingness must route to “request evidence” rather than being silently imputed.

- **Data-authority checks:** determine whether the caller was authorized to use the evidence for the proposed purpose.
- **PII and sensitive-attribute classification:** minimize transmission; redact, tokenize, or use field-level policies.
- **Prompt/state-injection defense:** treat retrieved unstructured content as data, never authority. Documents must not be able to alter policy, tool permissions, identities, or objectives.
- **Candidate-set provenance:** in ranking/recommendation workflows, log *how alternatives entered and exited the set*. Manipulating the candidate set is often more consequential than manipulating the scorer.

A critical extension for Jev specifically is an **input lineage graph**. Since Jev evaluates the state it receives, the decision firewall must be able to answer: *which upstream retrieval, transformation, human entry, or agent produced each material state field?*

3. Policy-as-code gate

Policy must sit outside the model and must not be editable through model output. It should be deterministic, versioned, testable, and signed.

The policy engine should produce one of:

- ALLOW
- ALLOW_WITH_CONSTRAINTS
- REQUIRE_EVIDENCE
- REQUIRE_REVIEW
- DENY
- STOP_AND_COMPENSATE for post-execution anomaly cases

Illustrative policy logic:

```
default decision := "REQUIRE_REVIEW"

decision := "DENY" if {
  input.proposed_action.action_type == "create_purchase_order"
  input.incentives.sponsorship_status == "undisclosed"
}

decision := "REQUIRE_EVIDENCE" if {
  not input.evidence.completeness.tax_status_verified
}

decision := "REQUIRE_REVIEW" if {
  input.risk.impact >= 0.7
  input.model_assessment.confidence < 0.9
}

decision := "ALLOW_WITH_CONSTRAINTS" if {
  input.model_assessment.confidence >= 0.85
}
```

```
input.policy_context.execution_authority_requested == "purchase-request-create"
input.proposed_action.amount.value <= input.actor.authority_limit
}

decision := "ALLOW" if {
  input.proposed_action.action_type == "read_only_recommendation"
}
```

This is the conceptual inversion that matters:

A high probability is evidence; it is not permission.

The policy engine should also evaluate the **composed workflow**, not merely the individual Jev result. For example:

- Whether a decision threshold was changed from 0.85 to 0.61.
- Whether a model is invoked only on cases expected to yield favorable outcomes.
- Whether the workflow automatically executes favorable decisions but reviews unfavorable ones.
- Whether several individually low-risk decisions combine into a high-impact lifecycle decision.
- Whether an action is reversible and whether a rollback mechanism exists.

4. Authorization-bound execution

The most important technical missing layer is an **execution capability broker**.

The firewall must issue a short-lived, cryptographically bound authorization token that includes:

- Decision ID and decision receipt hash.
- Exact tool/action type.
- Exact target resource or approved entity.
- Material action parameters, or a canonical parameters hash.
- Policy and approval version.
- Actor identity and delegation chain.
- Expiry time.
- One-time nonce and replay protection.
- Permitted side-effect scope and spend/impact limits.
- Required human approver, when relevant.

The execution adapter must reject requests that lack a valid authorization or materially differ from the authorized intent.

That prevents the classic approval-binding failure:

Approved: purchase Vendor A for \$18,500
Executed: purchase Vendor B for \$27,500

The approval must not be an ambient “yes.” It must be an authorization for *that exact action*, under *that decision state*, before *that expiry time*.

For agentic systems, this becomes a zero-trust model:

- Agents never hold unrestricted production credentials.
- Agents request a scoped capability from the firewall.
- The firewall evaluates context and policy.
- The broker issues a constrained, expiring capability.
- The target adapter validates it and emits an execution event.
- All paths—including direct API paths—must be covered or detected as bypasses.

5. Human oversight as a designed workflow

“Human-in-the-loop” is not a control unless the reviewer has authority, adequate context, enough time, and a usable escalation outcome.

A reviewer should receive:

- The proposed action, impact and reversibility.
- The exact question sent to Jev and the returned distribution/confidence.
- Decision workflow version and threshold.
- Evidence sources, freshness, missing fields and contradictory items.
- Policy checks that passed, failed or were waived.
- Objective declaration and disclosed incentives/conflicts.
- Counterfactual/sensitivity results.
- Independent-judge disagreement, if configured.
- A bounded choice: approve, reject, request evidence, select a different alternative, override with reason, or pause.

Record reviewer identity, decision, rationale, time-to-review, override reason and downstream outcome. This enables later audit of whether humans are functioning as a real safeguard or merely rubber-stamping machine recommendations.

Missing governance capabilities

Your source material correctly identifies receipts, provenance, counterfactual testing, shadow judges, incentive auditing, and outcome monitoring. I would extend this into ten distinct governance services.

Governance service	Why it is needed	Concrete mechanism
Decision inventory	You cannot govern undiscovered decision points	Register every workflow, decision type, model/provider, action class, data class and accountable owner
Risk classification	Same confidence has different meaning for different harms	Classify by impact, reversibility, affected population, autonomy, legal domain, data sensitivity and externality
Objective registry	Bias often enters as an unstated optimization target	Version objective functions, weights, prohibited objectives, business owner, and user-facing claim
Policy registry	Decision policy must be auditable and reproducible	Signed policy bundles, change approvals, simulation before rollout, rollback
Authority and delegation graph	Accountability otherwise diffuses across model, platform and user	Map who can define objectives, change thresholds, approve exceptions and execute actions
Incentive/conflict registry	Commercial interests can be hidden in inputs, candidate sets or ranking logic	Mandatory relationship declarations, sponsored-feature annotations, commission fields, vendor-affiliation graph
Counterfactual test service	Direct explanations are weak evidence of causal behavior	Paired tests, protected/proxy attribute sensitivity, candidate-set perturbations, threshold and feature sensitivity
Independent assurance	Self-certification creates a conflict of interest	Rules checks, independent model/judge, domain validator, periodic third-party evaluation
Appeals and redress	Rejected/filtered users otherwise disappear from feedback loops	Decision notice, appeal route, review SLA, override process, correction and compensation workflow
Change management	Many governance failures are deployment changes, not model failures	Diff model/question/state schema/policy/threshold/candidate source; canary, shadow, rollback and approval gates

Incentive integrity needs its own layer

This is the least developed but most differentiating part of the thesis.

A decision firewall should not claim it can infer hidden intent from a model's probability. It should instead create **testable accountability** through disclosure, structural separation, and anomaly detection.

Build an Objective and Incentive Registry:

```

Decision endpoint: hotel_ranker_v7
User-facing claim: "Best match for your trip"
Declared optimization:
  0.55 user suitability
  0.25 total price
  0.20 availability reliability

Commercial signals:
  sponsored placement: prohibited in "best match"
  affiliate commission: must be excluded
  partner inventory: permitted only in a separately labelled view

Owner: travel-ranking-team

```

Business approver: marketplace-director
Policy: ranking-integrity/2026-09

Then test:

- Does adding or removing a commercial relationship materially change selection while all user-relevant variables are fixed?
- Does the winner change when the candidate set includes non-partner alternatives?
- Do commercial features correlate with favorable outcomes beyond their declared role?
- Did a policy or model release cause sponsored/partner entities to win disproportionately?
- Are “best”, “recommended”, “safest”, or “lowest cost” claims aligned with the registered objective?

A sponsorship perturbation is an **investigation signal**, not proof of corruption. The robust claim is not “we can read the model’s motivations”; it is “we can ensure that incentives are declared, controls are enforced, unexpected sensitivity is detected, and claims can be independently audited.”

Observability that matters

Traditional LLM observability focuses on latency, token usage, traces, tool calls, errors, and optionally quality. You need all of that, but decision observability must be structured around the lifecycle:

intent → evidence → judgment → permission → execution → outcome

Decision receipt

Create a tamper-evident, append-only **Decision Receipt** at every consequential decision. Do not store raw secrets or unnecessary PII in the receipt; store encrypted evidence references, content hashes, access controls and retention metadata.

Minimum receipt fields:

Identity

- decision_id, tenant, workflow, run, correlation and causation IDs
- actor identity, agent identity, delegated authority chain

Decision

- decision type, question/state schema versions, alternatives, selected outcome
- provider/model/version, model configuration, probabilities/confidence
- workflow graph/version and code/configuration hash

Evidence

- source references, timestamps, integrity hashes, lineage
- field classification: observed, inferred, supplied, generated, missing
- retrieval/query/transformation metadata

Governance

- declared objective and weights

- incentive/conflict disclosures
- risk score and risk-tier decision
- policy bundle/version, results, exceptions and override reasons
- independent-model/rules/human-review results

Execution

- requested capability and canonicalized parameters hash
- authorization token ID, policy decision, approver identity
- execution response, idempotency key and compensating action

Outcome

- realized outcome events
- actual cost/benefit, appeal, reversal, harm report
- labels for calibration and fairness evaluation where lawful and appropriate

Use a hash chain or Merkle-tree approach with periodic signed checkpoints. That does not make the input truthful, but it makes post-hoc alteration detectable. For regulated or high-consequence workflows, use write-once retention semantics, access logging, separation of duties, and explicit retention/deletion policies.

Operational metrics

Track model and governance signals separately.

Metric family	Examples	Why it matters
Availability and performance	p50/p95/p99 decision latency, timeout rate, error rate, gate latency	A firewall that breaks critical workflows becomes a bypass incentive
Decision distribution	Choice rate, score distribution, entropy, abstention rate, runner-up margin	Detect silent behavioral shifts
Calibration	Brier score, expected calibration error, reliability diagrams by cohort and action	Confidence must be validated against outcomes, not trusted blindly
Policy behavior	Allow/deny/escalate rate, policy failure reasons, exception rate, stale-evidence blocks	Shows whether policy is actually controlling execution
Human oversight	Review rate, time to review, override rate, agreement/disagreement, reviewer consistency	Detect rubber-stamping and bottlenecks
Execution integrity	Unauthorized attempts, invalid/replayed tokens, parameter mismatch, bypass detections	Measures whether the firewall is real
Outcome quality	Reversal rate, appeal rate, false-positive/false-negative rate, complaint rate, realized cost/benefit	Connects predictions to consequences
Fairness and disparity	Selection/approval rate, calibration, error rates and override rates across approved cohorts	Detect uneven distribution of errors and burdens
Incentive integrity	Partner win-rate, sponsorship sensitivity, commission correlation, candidate-set composition	Detect possible objective leakage
Change impact	Pre/post-release deltas for decision rates, thresholds, outcomes and subgroup behavior	Catches policy/model/data drift

A confidence value should never be the top-level risk metric. Use a composite execution-risk calculation:

$$R_{\text{exec}} = f(I, U, A, V, E, S, C)$$

Where:

- *I*: impact or harm severity if wrong.
- *U*: uncertainty, including low confidence or small decision margin.
- *A*: autonomy and irreversibility of the action.
- *V*: vulnerability to incentives, manipulation, or strategic gaming.
- *E*: evidence quality, freshness, completeness and provenance.
- *S*: sensitivity of the affected domain/population.
- *C*: control coverage, including independent checks and review availability.

An action can be high-risk even when Jev is 94% confident—for example, a non-reversible denial of credit, benefit, access, or employment opportunity.

Security architecture additions

A firewall around decision models also needs an AI-specific threat model. The meaningful attack surface is not just the model endpoint.

Threats to explicitly design for

- **State/context poisoning:** manipulate CRM fields, retrieval documents, evaluation inputs, or upstream ETL so the model sees a skewed state.
- **Question or schema manipulation:** redefine an atomic question or rubric so a formally correct answer serves a different business purpose.
- **Candidate-set manipulation:** exclude superior alternatives before selection; especially relevant in vendor choice, ranking and recommendation.
- **Threshold laundering:** keep model/model-card unchanged but quietly lower the action threshold.
- **Selective invocation:** call the model only where a preferred answer is expected, creating hidden selection bias.
- **Policy injection or configuration drift:** let agents, prompts, or tenant configuration mutate policies or their interpretation.
- **Tool/capability escalation:** turn a valid recommendation permission into broad production access.
- **Approval replay and TOCTOU:** reuse a previously approved token or alter evidence/action after approval but before execution.
- **Audit evasion:** suppress traces, use shadow endpoints, execute side effects via an unregistered API, or mutate receipts.

- **Feedback poisoning:** contaminate downstream outcome labels, especially where accepted and rejected populations have different observability.
- **Decision monoculture:** correlated failure across many services using the same model, workflow library, or policy defect.

Required security controls

- Strong workload identity: SPIFFE/SVID, mTLS, or equivalent service identity.
- Separate identities for model caller, policy engine, reviewer, execution adapter and auditor.
- Least-privilege, short-lived capability tokens, not standing credentials in agents.
- Signed model/version/config/policy manifests.
- Idempotency keys and replay prevention for every side-effecting operation.
- Evidence hash binding and optimistic concurrency/version checks.
- Separate control plane from data plane; separate policy authoring from policy deployment.
- Audit-store immutability and independent export/retention path.
- Network/service mesh egress controls so tool use cannot bypass the broker.
- Shadow mode, canary releases, kill switches and compensating-action playbooks.
- Continuous adversarial testing: indirect prompt injection, state poisoning, authorization mutation, stale-evidence, token replay, bypass routes and candidate-set attacks.

Standards and regulatory alignment

The framework should map cleanly to existing governance language so it is enterprise-adoptable rather than a new isolated taxonomy.

NIST AI RMF organizes AI risk management as **Govern, Map, Measure, and Manage**. That gives a useful top-level operating model for the decision firewall: define accountability and policies; map decision contexts and impacts; measure performance, drift and impacts; then control, respond, and improve. ^[4] ^[5] ^[6]

NIST AI RMF function	Decision Firewall implementation
Govern	Ownership, authority graph, objective registry, risk taxonomy, policy lifecycle, assurance evidence
Map	Decision inventory, stakeholder/impact mapping, workflow graph, evidence lineage, threat model, use-case classification
Measure	Calibration, reliability, bias/disparity, counterfactual sensitivity, policy conformance, red-team results, outcome analysis
Manage	Runtime allow/deny/escalate control, scoped authorization, incident response, rollback, review, appeal and post-market monitoring

For EU-facing deployments, this architecture is also materially aligned with the direction of the EU AI Act. The Commission identifies certain employment, credit/access-to-essential-services, education, critical-infrastructure, biometric, border, justice and public-sector uses as high risk.

The framework calls for risk management, data quality, activity logging/traceability, documentation, human oversight, robustness, cybersecurity, accuracy, post-market monitoring and incident reporting.^[7]

That does **not** mean a generic firewall by itself creates compliance. It means the receipts, evidence lineage, policy records, human-oversight workflow, execution controls, outcome monitoring, and change history form a strong compliance evidence substrate. Legal classification remains use-case-, jurisdiction-, provider-, and deployer-specific.

Product scope and differentiation

Do not start by building a full enterprise governance suite. Existing governance tools already address inventories, model registries, documentation workflows and compliance dashboards. The sharp wedge is **runtime decision enforcement plus outcome-linked auditability**.

MVP: build this first

Start with AI-assisted vendor selection and purchase-request approval, as suggested in the attached material. It exercises incentives, alternative sets, evidence provenance, monetary authority, human approval and observable downstream outcome without beginning in an immediately regulated life-impact domain.^[8]

Build these six components:

1. Decision SDK and schema

- Provider-neutral decision envelope.
- Native Jev adapter first; adapters for a rules engine and one LLM-based judge second.
- OpenTelemetry-compatible trace/correlation identifiers.

2. Evidence registry

- Source references, hashes, freshness, classification, candidate-set lineage.
- Explicit unknown/missing evidence states.

3. Policy and risk engine

- Versioned policy-as-code.
- Rules for evidence requirements, spend bands, conflicts, threshold/margin requirements, action reversibility and escalation.

4. Authorization-bound execution adapter

- Only permits exact, short-lived, signed purchase-request capabilities.
- Enforces amount, supplier, approver, expiry and parameter binding.

5. Decision inspector and receipt store

- Decision timeline: state → judgment → policy → approval → action → outcome.
- Replay/simulation against a historic receipt using a policy version.

6. Outcome and integrity dashboard

- Override/reversal metrics, policy violations, decision latency, evidence quality, vendor win-rates and conflict/sponsorship signals.

Explicit non-goals for v1

- Claiming to recover a model's hidden chain of thought.
- Certifying fairness or lack of commercial bias from a single decision.
- Replacing procurement, legal, security or risk owners.
- Building a generic "AI ethics dashboard" with no runtime enforcement.
- Automatically deciding human-review outcomes.
- Supporting unrestricted autonomous execution.

The strongest product thesis

AI models can form judgments. A Decision Firewall determines whether those judgments have sufficient evidence, authority, policy compliance, and accountability to become real-world actions.

Or in Jev-specific terms:

Jev can return a typed decision with probability and confidence. The Decision Firewall turns that output into a governed proposal, independently determines whether it may execute, and preserves the evidence needed to explain the action and evaluate its consequences later.

That is the layer Jev deliberately does not attempt to be—and it is where the highest-value governance, security, audit, and observability work belongs.



1. [info.md](#)
2. <https://docs.typesafe.ai/introduction>
3. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>
4. <https://airc.nist.gov/airmf-resources/airmf/>
5. <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
6. <https://www.nist.gov/itl/ai-risk-management-framework/nist-ai-rmf-playbook>
7. <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
8. [info-2.md](#)
9. <https://arxiv.org/abs/2510.25863>
10. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
11. <https://ieeexplore.ieee.org/document/11275565/>
12. <https://journal.uitm.edu.my/index.php/JIKM/article/view/8665>
13. <https://hstalks.com/doi/10.69554/VBLF1901/>

14. https://airc.nist.gov/docs/AI_RM_F_Playbook.pdf
15. <https://digitalcommons.kennesaw.edu/jcerp/vol2026/iss1/18>
16. <https://typesafe.ai/>
17. <https://www.spiedigitallibrary.org/conference-proceedings-of-spie/14046/3100740/Operationalizing-enterprise-AI-security--an-open-source-framework-for/10.1117/12.3100740.full>
18. <https://csecurity.kubg.edu.ua/index.php/journal/article/view/993>
19. <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/IJIT.367471>
20. <https://docs.typesafe.ai/models>
21. <https://journal.sinergi.or.id/index.php/ijis/article/view/897>
22. <https://artificialintelligenceact.eu/article/14/>
23. <https://artificialintelligenceact.eu/high-level-summary/>
24. <https://artificialintelligenceact.eu/chapter/3/>
25. <https://aijev.org/>
26. <https://efros.com/resources/nist-ai-rmf-implementation-guide/>
27. <https://www.ijisrt.com/shipping-safer-llm-features-in-smes-a-onepage-checklist-mapped-to-nist-ai-rmf-and-isoiec-2389442001>